

FasoShield

Antivirus mobile national

Feuille de route et architecture

Version	1.0 — 23 juillet 2026
Auteur	Cedric Severin DJIGUIMDE
Statut	Phases 1 et 2 livrées (moteur + API, 34 tests, CI verte) — phase 3 en cours
Dépôt	github.com/CedricCloudOps/fasoshield

Plateforme souveraine de protection contre les menaces mobiles : moteur d'analyse d'APK, réputation de fichiers, distribution de signatures nationales et agent Android grand public, conçus pour le paysage de menaces ouest-africain (fraude mobile money, vol d'OTP, smishing).

1. Contexte et menace

Le paiement mobile est l'infrastructure financière dominante en Afrique de l'Ouest : les comptes Orange Money, Moov Money ou Wave y tiennent le rôle des comptes bancaires. Cette concentration attire des campagnes de fraude industrialisées, largement diffusées hors des canaux officiels (liens WhatsApp et Telegram, boutiques d'applications alternatives).

Vecteur	Technique	Impact
Fausse applications financières	Clones d'applications mobile money diffusés hors store ; l'application capture le code PIN et l'identifiant du compte.	Vidage direct du compte de la victime.
Vol d'OTP par interception SMS	Malware demandant RECEIVE_SMS / READ_SMS pour capter les codes de validation, puis exfiltration HTTP vers un serveur de commande.	Validation frauduleuse de transactions à l'insu de la victime.
Smishing	SMS en français usurpant l'opérateur (compte suspendu, retrait en attente), pointant vers un kit de phishing ou un APK malveillant.	Collecte d'identifiants et propagation des deux premiers vecteurs.

Pourquoi une solution souveraine

- Les antivirus commerciaux ciblent les menaces globales : pas de registre des applications financières locales, pas de règles sur les leurres en français.
- La base de signatures est alimentée par le CERT national et les opérateurs locaux, au plus près des campagnes réellement observées.
- Les données (échantillons, télémétrie) restent hébergées sur le territoire, avec une télémétrie anonymisée par construction.

2. Architecture cible

Trois familles de clients consomment une API unique. Les agents mobiles utilisent en priorité le chemin léger (réputation par hash) pour économiser les forfaits de données ; l'upload complet d'APK est réservé aux fichiers inconnus et aux analystes.

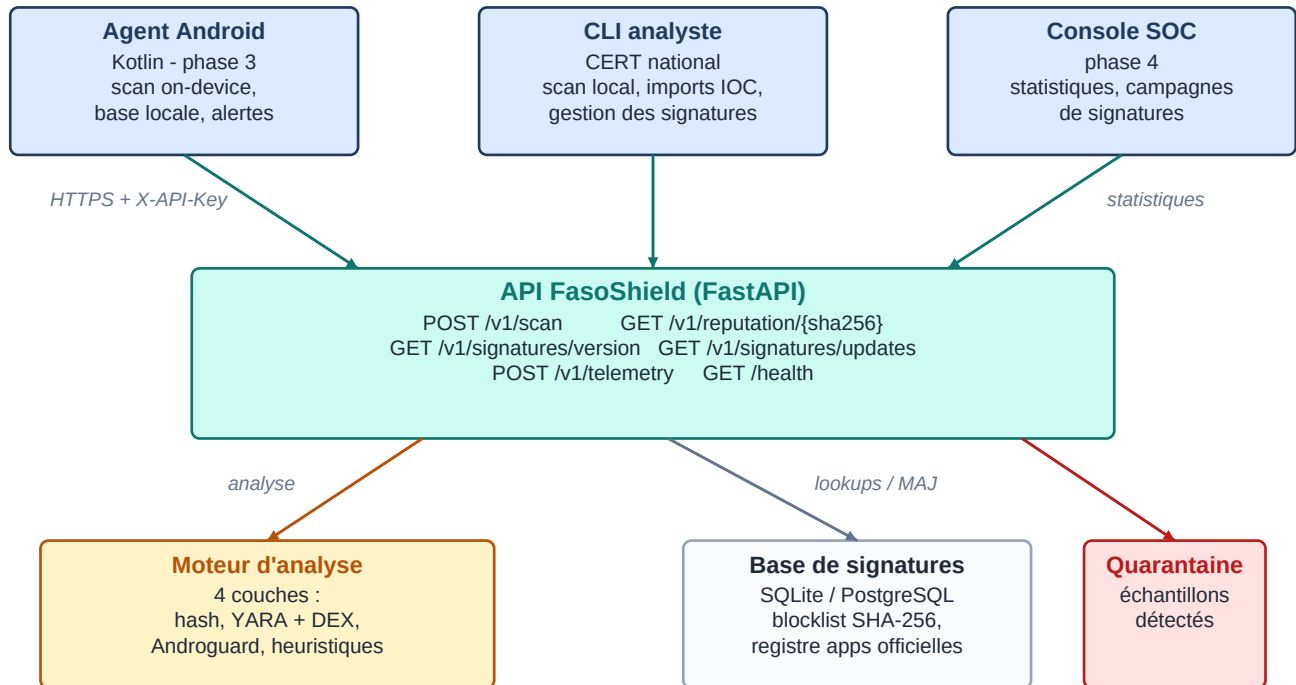


Figure 1 — Architecture d'ensemble de la plateforme

Endpoint	Rôle	Consommateur
POST /v1/scan	Analyse complète d'un APK soumis ; quarantaine si détection	Agents, analystes
GET /v1/reputation/{sha256}	Verdict immédiat sans upload (chemin chaud)	Agents mobiles
GET /v1/signatures/version	Version courante de la base de signatures	Agents mobiles
GET /v1/signatures/updates	Synchronisation delta de la blocklist embarquée	Agents mobiles
POST /v1/telemetry	Événements de détection anonymisés	Agents mobiles

3. Pipeline d'analyse

Le moteur enchaîne quatre couches, de la moins coûteuse à la plus coûteuse. Chaque couche produit des constats (findings) avec une sévérité ; le verdict final agrège les sévérités en un score de 0 à 100. Un constat CRITICAL (signature connue, usurpation de certificat) classe à lui seul l'échantillon MALICIOUS. Le pipeline aboutit toujours : un fichier corrompu ou non-APK produit quand même un rapport (hash + YARA).

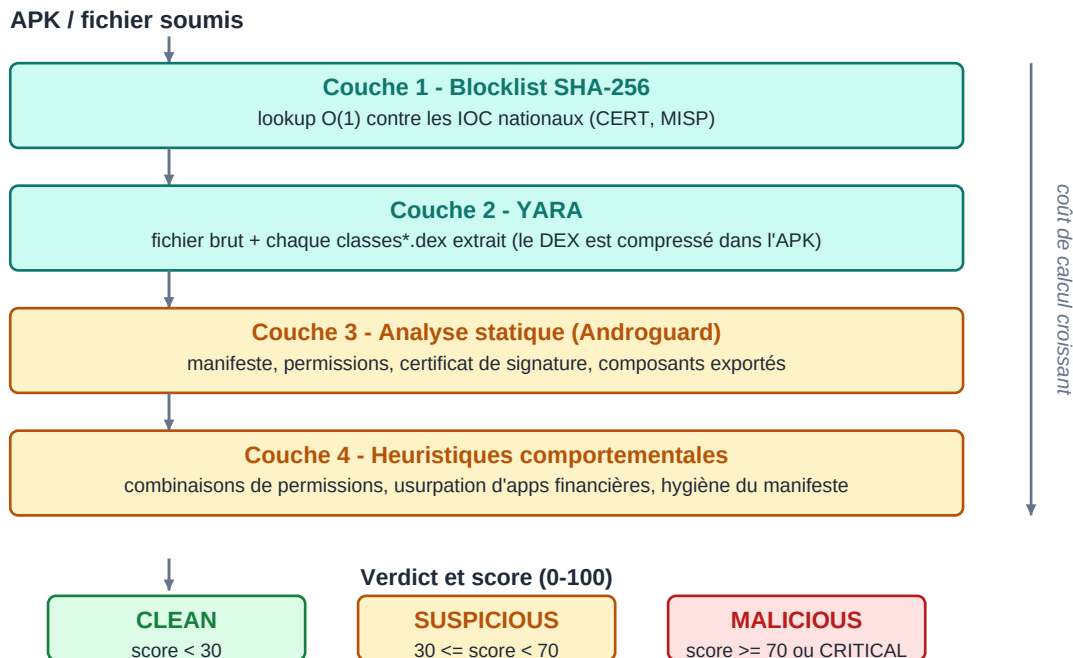


Figure 2 — Les quatre couches du moteur et l'échelle de verdict

Point technique déterminant

Dans un APK, classes.dex est compressé (DEFLATE) : un scan YARA du fichier brut ne voit jamais les chaînes du bytecode. Le moteur extrait donc chaque entrée classes*.dex et la scanne en mémoire, en plus du scan du conteneur. Les résultats sont dédoublonnés par règle.

4. Feuille de route



Figure 3 — Avancement des cinq phases

Phase	Livrables	Critère de sortie	Statut
1. Moteur d'analyse	Pipeline 4 couches, CLI analyste (codes de sortie shell), 6 règles YARA, bases seed (EICAR, registre officiel)	Suite de tests verte ; détection EICAR bout-en-bout	Livrée
2. API plateforme	5 endpoints, auth par clé d'API agent, SQLite dev / PostgreSQL prod, quarantaine, CI GitHub Actions (lint, tests, SAST, audit)	Upload EICAR → MALICIOUS → réputation servie depuis l'historique ; pip-audit à zéro	Livrée
3. Agent Android	Kotlin, minSdk 24 : scan des apps installées, détection des nouvelles installations, base Room synchronisée en delta, alertes utilisateur, mode hors-ligne complet	Détection locale d'un APK de test sans réseau, télémétrie remontée à la reconnexion	En cours
4. Console SOC	Tableau de bord des détections, cycle de vie des signatures (proposition → revue → publication), exports MISP/STIX	Publication d'une signature de bout en bout depuis la console	À venir
5. Durcissement	Audit externe, signature APK par l'autorité nationale, Play Store + canal officiel, DPIA, montée en charge (file de scan asynchrone)	Audit passé ; déploiement pilote validé	À venir

5. Synchronisation des agents mobiles

L'agent embarque une copie locale de la blocklist (base Room) et fonctionne entièrement hors-ligne. À chaque connexion, il compare sa version locale à celle du serveur et ne télécharge que le delta — la version de la base est un horodatage UTC strictement croissant, incrémenté à chaque import d'IOC.

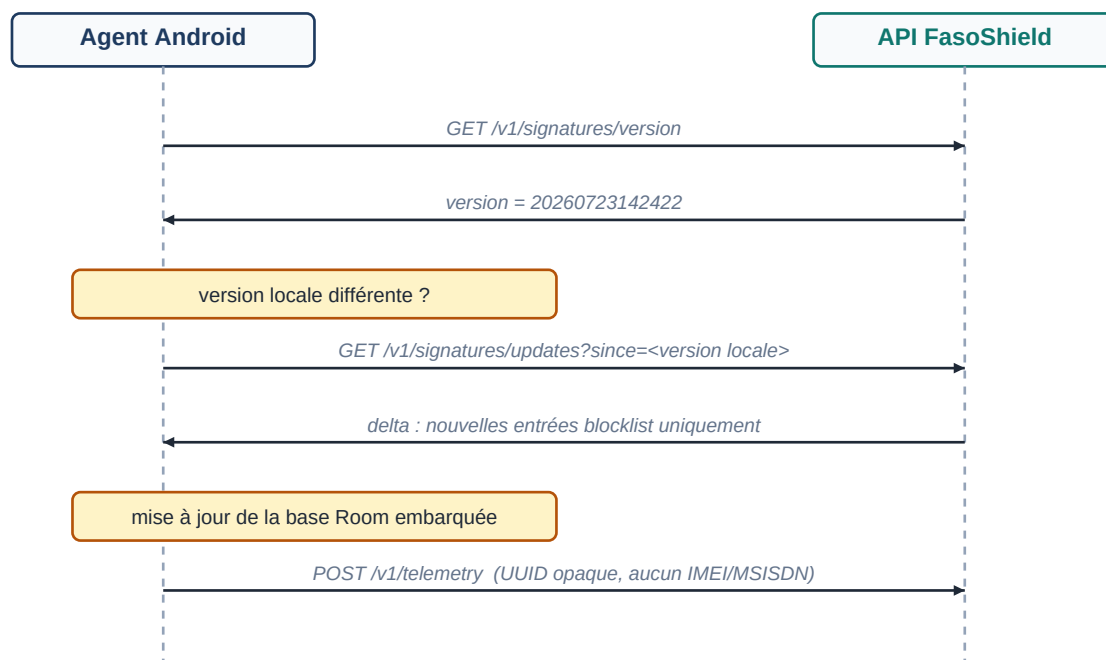


Figure 4 — Séquence de synchronisation delta et télémétrie

Confidentialité par construction

- L'agent s'identifie par un UUID opaque auto-généré : aucun IMEI, MSISDN ou identifiant de compte ne transite ni n'est stocké.
- La région remontée est déclarative et grossière (échelle régionale), suffisante pour la cartographie nationale des campagnes.
- Le schéma de télémétrie rend la réidentification impossible côté plateforme : c'est un argument de conformité, pas une promesse.

6. Risques et mitigations

Risque	Impact	Mitigation
Faux positifs sur applications légitimes	Perte de confiance des utilisateurs et des éditeurs	Registre officiel avec épingleage de certificat, seuils conservateurs, revue humaine avant entrée en blocklist
Contournement par obfuscation du DEX	Détection YARA dégradée	Couche heuristique indépendante du contenu DEX (permissions, certificat, manifeste)
Fuite de la base de signatures	Les attaquants testent leurs APK avant diffusion	Clés d'API par agent, distribution delta, rotation des clés
Données personnelles dans la télémétrie	Risque juridique et réputationnel	Anonymisation à la source, schéma sans identifiant direct, DPIA en phase 5

Prochaines étapes immédiates

- Phase 3 : agent Android Kotlin (scaffold du projet, scan on-device, synchronisation Room, alertes).
- Validation du moteur sur des APK réels et épingleage des certificats officiels dans le registre.
- Préparation de la console SOC (phase 4) sur la base des tables de télémétrie existantes.